ONEDRAFT

CAD-AI — Aria Powered

Integration & Event Bus

Version 0.1

Status: Implementation Specification
Architecture: Event-Driven Integration Layer
Messaging: Asynchronous Event Bus
API: REST / JSON + Internal Events
Identifiers: UUID
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered

---

1. INTEGRATION & EVENT BUS PRINCIPLE

The OneDraft Integration & Event Bus provides the communication layer between the independently executing subsystems of the platform.

It connects:

Project Services

Geometry Engine

Parametric Model Engine

Compliance Engine

Validation Engine

Revision System

Documentation Engine

Runtime Orchestrator

Job & Worker System

Artifact Storage

Aria Intelligence

External Integrations

without requiring those systems to directly depend upon one another's internal implementation.

The event bus therefore becomes the asynchronous nervous system of OneDraft.

---

2. ARCHITECTURAL POSITION

The Integration & Event Bus sits between the application services and execution infrastructure:

```
CLIENT
  ↓
API
  ↓
DOMAIN SERVICES
  ↓
EVENT BUS
  ↓
┌──────────────────────────────┐
│ PROJECT                      │
│ MODEL                        │
│ GEOMETRY                     │
│ COMPLIANCE                   │
│ VALIDATION                   │
│ DOCUMENTATION                │
│ ARIA                         │
│ RUNTIME                      │
│ WORKERS                      │
│ ARTIFACT STORAGE             │
└──────────────────────────────┘
```

No subsystem should require synchronous knowledge of the internal implementation of another subsystem merely to announce that an operation has occurred.

---

3. EVENT-DRIVEN DESIGN

OneDraft shall use events to communicate completed or materially changed system state.

Examples include:

```
PROJECT_CREATED
MODEL_VERSION_CREATED
ELEMENT_CREATED
ELEMENT_MODIFIED
CONSTRAINT_CREATED
ARIA_COMMAND_PROPOSED
ARIA_COMMAND_EXECUTED
GEOMETRY_GENERATED
VALIDATION_COMPLETED
REDLINE_CREATED
REDLINE_RESOLVED
```

```
DRAWING_GENERATED
DOCUMENT_PACKAGE_GENERATED
CUSTOMER_ACCEPTED
```

Events communicate facts.

Commands request actions.

These concepts must remain distinct.

---

4. COMMAND / EVENT DISTINCTION

A command represents an instruction:

```
MOVE_ELEMENT
GENERATE_DRAWINGS
RUN_VALIDATION
GENERATE_DOCUMENT_PACKAGE
RESOLVE_REDLINE
```

An event represents a completed or recorded fact:

```
ELEMENT_MOVED
DRAWINGS_GENERATED
VALIDATION_COMPLETED
DOCUMENT_PACKAGE_GENERATED
REDLINE_RESOLVED
```

The system must never treat an event as an implicit command.

---

5. EVENT BUS RESPONSIBILITIES

The event bus shall provide:

Event publication

Event delivery

Event routing

Event persistence

Event ordering where required

Retry handling

Dead-letter handling

Consumer acknowledgement

Idempotent consumption

Correlation

Causation tracking

Schema validation

Observability

Integration boundaries

---

6. EVENT SCHEMAS

All OneDraft events shall use a common envelope.

Conceptually:

```
{
  "event_id": "uuid",
  "event_type": "MODEL_VERSION_CREATED",
  "event_version": 1,
  "occurred_at": "ISO-8601",
  "project_id": "uuid",
  "organization_id": "uuid",
  "actor_id": "uuid",
  "correlation_id": "uuid",
  "causation_id": "uuid",
  "source": "project_model_service",
  "payload": {}
}
```

The envelope is independent from the event payload.

---

7. EVENT ID

Every event receives a globally unique:

`event_id`

The event identifier shall never be reused.

Consumers use the event identifier to support idempotent processing.

---

8. CORRELATION ID

Every related execution chain shall carry a:

```
correlation_id
```

Example:

```
ARIA REQUEST
 ↓
COMMAND
 ↓
MODEL MUTATION
 ↓
GEOMETRY JOB
 ↓
VALIDATION
 ↓
DRAWING GENERATION
 ↓
DOCUMENT PACKAGE
```

All events resulting from that execution chain may share the same correlation identifier.

This allows the complete lifecycle of a user operation to be reconstructed.

---

9. CAUSATION ID

Every event generated as a consequence of another event or command should identify its immediate cause through:

```
causation_id
```

Example:

```
ARIA_COMMAND_EXECUTED
        ↓
MODEL_VERSION_CREATED
```

The second event records the first event or command as its cause.

---

10. EVENT SOURCE

Every event identifies the subsystem that generated it.

Examples:

```
api
project_model_service
geometry_engine
compliance_engine
validation_engine
documentation_engine
runtime_orchestrator
```

```
worker
aria
artifact_service
acceptance_service
```

This establishes event provenance.

---

11. EVENT VERSIONING

Events are versioned independently from the API.

Example:

```
MODEL_VERSION_CREATED.v1
MODEL_VERSION_CREATED.v2
```

A consumer must explicitly support the event versions it understands.

Breaking payload changes require a new event version.

---

12. EVENT NAMING

Events shall use:

```
UPPER_SNAKE_CASE
```

Examples:

```
PROJECT_CREATED
PROJECT_UPDATED
PROJECT_ARCHIVED

BUILDING_CREATED
LEVEL_CREATED
SPACE_CREATED
ELEMENT_CREATED
ELEMENT_MODIFIED
ELEMENT_DELETED

MODEL_VERSION_CREATED
REVISION_CREATED

ARIA_COMMAND_PROPOSED
ARIA_COMMAND_VALIDATED
ARIA_COMMAND_EXECUTED
ARIA_COMMAND_FAILED

GEOMETRY_JOB_QUEUED
GEOMETRY_GENERATED
GEOMETRY_INVALIDATED
```

```
VALIDATION_STARTED
VALIDATION_COMPLETED
VALIDATION_FAILED

DRAWING_GENERATION_STARTED
DRAWING_GENERATED
DRAWING_INVALIDATED

DOCUMENT_PACKAGE_GENERATION_STARTED
DOCUMENT_PACKAGE_GENERATED

REDLINE_CREATED
REDLINE_IMPLEMENTED
REDLINE_REJECTED

CUSTOMER_ACCEPTANCE_RECORDED
```

---

13. EVENT CATEGORIES

Events are grouped into:

```
IDENTITY
PROJECT
MODEL
GEOMETRY
ARIA
COMPLIANCE
VALIDATION
REVISION
DOCUMENTATION
REDLINE
JOB
ARTIFACT
ACCEPTANCE
INTEGRATION
SYSTEM
AUDIT
```

This classification is used for routing and observability.

---

14. PROJECT EVENTS

Initial project events:

```
PROJECT_CREATED
PROJECT_UPDATED
PROJECT_ARCHIVED
PROJECT_RESTORED
PROJECT_STATUS_CHANGED
```

Each event contains the relevant project identifier and state metadata.

---

15. MODEL EVENTS

Initial model events:

```
MODEL_CREATED
MODEL_VERSION_CREATED
MODEL_VERSION_ACTIVATED
MODEL_VERSION_SUPERSEDED

ELEMENT_CREATED
ELEMENT_MODIFIED
ELEMENT_DELETED

RELATIONSHIP_CREATED
RELATIONSHIP_MODIFIED
RELATIONSHIP_DELETED

CONSTRAINT_CREATED
CONSTRAINT_MODIFIED
CONSTRAINT_REMOVED
```

---

16. GEOMETRY EVENTS

Initial geometry events:

```
GEOMETRY_JOB_QUEUED
GEOMETRY_JOB_STARTED
GEOMETRY_GENERATED
GEOMETRY_VALIDATED
GEOMETRY_INVALIDATED
GEOMETRY_JOB_FAILED
```

Geometry events must identify the model version from which the geometry was generated.

---

17. COMPLIANCE EVENTS

Initial compliance events:

```
CODE_SOURCE_REGISTERED
CODE_SOURCE_UPDATED

CODE_MANIFEST_CREATED
CODE_MANIFEST_ACTIVATED
CODE_MANIFEST_SUPERSEDED

CODE_RULE_UPDATED
```

A finalized code manifest is immutable.

---

18. VALIDATION EVENTS

Initial validation events:

```
VALIDATION_REQUESTED
VALIDATION_STARTED
VALIDATION_COMPLETED
VALIDATION_FAILED
VALIDATION_RESULT_CREATED
VALIDATION_RESULT_RESOLVED
```

Validation events identify:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
```

where applicable.

---

19. DOCUMENTATION EVENTS

Initial documentation events:

```
VIEW_CREATED
VIEW_UPDATED

DRAWING_REQUESTED
DRAWING_GENERATION_STARTED
DRAWING_GENERATED
DRAWING_INVALIDATED

SHEET_CREATED
SHEET_UPDATED

SCHEDULE_GENERATED

DOCUMENT_PACKAGE_REQUESTED
DOCUMENT_PACKAGE_GENERATION_STARTED
DOCUMENT_PACKAGE_GENERATED
DOCUMENT_PACKAGE_INVALIDATED
```

---

20. REDLINE EVENTS

Initial redline events:

```
REDLINE_CREATED
REDLINE_ACCEPTED
REDLINE_REJECTED
REDLINE_IMPLEMENTATION_STARTED
REDLINE_IMPLEMENTED
REDLINE_VALIDATION_FAILED
REDLINE_RESOLVED
```

A redline becomes resolved only after the corresponding model operation and required validation have completed.

---

21. ARIA EVENTS

Aria operates through application capabilities rather than direct database access.

Initial events:

```
ARIA_REQUEST_RECEIVED
ARIA_COMMAND_PROPOSED
ARIA_COMMAND_VALIDATED
ARIA_COMMAND_APPROVED
ARIA_COMMAND_EXECUTED
ARIA_COMMAND_REJECTED
ARIA_COMMAND_FAILED
```

Aria-generated events must identify the command that produced them.

---

22. JOB EVENTS

The Job & Worker Execution System publishes:

```
JOB_CREATED
JOB_QUEUED
JOB_STARTED
JOB_PROGRESS_UPDATED
JOB_COMPLETED
JOB_FAILED
JOB_CANCELLED
JOB_RETRYING
JOB_DEAD_LETTERED
```

The event bus therefore connects job execution to the rest of the OneDraft architecture.

---

23. ARTIFACT EVENTS

The Artifact Storage subsystem publishes:

```
ARTIFACT_CREATED
ARTIFACT_VERSION_CREATED
ARTIFACT_HASHED
ARTIFACT_VALIDATED
ARTIFACT_ARCHIVED
ARTIFACT_DELETED
```

Artifact deletion must respect document provenance and retention requirements.

---

24. ACCEPTANCE EVENTS

Initial acceptance events:

```
ACCEPTANCE_REQUESTED
ACCEPTANCE_RECORDED
ACCEPTANCE_REVOKED
```

Acceptance records must reference the exact:

```
project
revision
model version
code manifest
validation state
document package
```

---

25. EVENT TOPICS

The initial logical topic structure shall be:

```
onedraft.project
onedraft.model
onedraft.geometry
onedraft.aria
onedraft.compliance
onedraft.validation
onedraft.revision
onedraft.documentation
onedraft.redline
onedraft.job
onedraft.artifact
onedraft.acceptance
onedraft.integration
onedraft.audit
onedraft.system
```

The physical messaging implementation may differ from the logical topic structure.

---

26. EVENT ROUTING

Consumers subscribe only to the events they require.

Example:

```
ELEMENT_MODIFIED
        ↓
geometry service
documentation service
validation service
audit service
```

The model service does not need to know which consumers exist.

This is a central decoupling property of the architecture.

---

27. EVENT PUBLISHER

A service publishes an event only after the corresponding state transition has been durably recorded.

The preferred sequence is:

```
BEGIN TRANSACTION

apply state change
record event in outbox

COMMIT

EVENT BUS PUBLISHER
        ↓
publish event
```

This avoids publishing events for transactions that subsequently roll back.

---

28. OUTBOX PATTERN

OneDraft shall use the transactional outbox pattern for important domain events.

Conceptually:

```
DOMAIN TRANSACTION
        ↓
DATABASE
```

```
  ├── STATE CHANGE
  └── OUTBOX EVENT
          ↓
OUTBOX PUBLISHER
          ↓
EVENT BUS
          ↓
CONSUMERS
```

This prevents the database and event bus from entering inconsistent states because of a failure between separate operations.

---

29. OUTBOX TABLE

The persistence layer shall contain:

```
system.event_outbox
```

Fields:

```
id UUID PRIMARY KEY
event_id UUID NOT NULL UNIQUE
event_type VARCHAR(128) NOT NULL
event_version INTEGER NOT NULL
aggregate_type VARCHAR(128)
aggregate_id UUID
project_id UUID
correlation_id UUID
causation_id UUID
payload JSONB NOT NULL
created_at TIMESTAMPTZ NOT NULL
published_at TIMESTAMPTZ
attempt_count INTEGER NOT NULL DEFAULT 0
status VARCHAR(32) NOT NULL
last_error TEXT
```

---

30. OUTBOX STATUS

Initial states:

```
PENDING
PUBLISHING
PUBLISHED
FAILED
DEAD_LETTERED
```

A published event is retained according to the platform's event-retention policy.

---

31. CONSUMER STATE

The system shall maintain consumer processing state.

Conceptually:

```
system.event_consumers
```

Fields:

```
id UUID PRIMARY KEY
consumer_name VARCHAR(255) NOT NULL
event_id UUID NOT NULL
status VARCHAR(32) NOT NULL
attempt_count INTEGER NOT NULL
processed_at TIMESTAMPTZ
last_error TEXT
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

32. IDEMPOTENT CONSUMPTION

Consumers must assume that an event may be delivered more than once.

Every consumer therefore implements:

```
CHECK
  ↓
PROCESS
  ↓
RECORD EVENT_ID
```

If the event identifier has already been processed, the consumer safely acknowledges the duplicate without repeating the state transition.

---

33. EVENT ORDERING

Global event ordering is not required.

Ordering is required only where domain correctness depends upon it.

For example:

```
MODEL_VERSION_CREATED
          ↓
MODEL_VERSION_ACTIVATED
```

must preserve ordering for the affected project/model stream.

Ordering may therefore be established by:

```
project_id
aggregate_id
```

rather than across the entire platform.

---

34. AGGREGATE STREAMS

Events should identify their aggregate.

Examples:

```
project_id
building_id
level_id
element_id
revision_id
job_id
document_package_id
```

An aggregate stream provides a logical ordering boundary.

---

35. EVENT RETENTION

Events associated with project provenance should be retained significantly longer than transient operational events.

At minimum, the architecture distinguishes:

```
DOMAIN EVENTS
AUDIT EVENTS
OPERATIONAL EVENTS
DEBUG EVENTS
```

Domain and audit events form part of project provenance.

---

36. EVENT REPLAY

The event architecture shall eventually support replay.

A historical event stream may be replayed to reconstruct derived state.

Example:

```
PROJECT_CREATED
```

```
 ↓
BUILDING_CREATED
 ↓
LEVEL_CREATED
 ↓
SPACE_CREATED
 ↓
ELEMENT_CREATED
 ↓
ELEMENT_MODIFIED
```

Replay must never silently modify the production source-of-truth state.

Replay initially occurs into:

```
rebuild
simulation
test
recovery
```

contexts.

---

37. EVENT REBUILD

Derived systems must be capable of rebuilding their state from authoritative project information and events where technically appropriate.

Examples include:

```
documentation indexes
search indexes
analytics
integration projections
notification state
```

The computational project model remains authoritative.

---

38. EVENT PAYLOAD PRINCIPLE

Events should contain sufficient information for consumers to process the event without requiring unnecessary synchronous calls to the publisher.

However, events should not become uncontrolled copies of the entire project model.

The payload should contain:

```
event facts
identifiers
state transition metadata
required derived information
```

Large computational payloads remain in the appropriate subsystem or artifact store.

---

39. EVENT PAYLOAD EXAMPLE

Example:

```
{
  "event_id": "uuid",
  "event_type": "ELEMENT_MODIFIED",
  "event_version": 1,
  "occurred_at": "2026-08-10T00:00:00Z",
  "project_id": "uuid",
  "correlation_id": "uuid",
  "causation_id": "uuid",
  "payload": {
    "element_id": "uuid",
    "revision_id": "uuid",
    "model_version_id": "uuid",
    "element_type": "WALL",
    "change_type": "GEOMETRY_AND_PARAMETER"
  }
}
```

---

40. EVENT SCHEMA REGISTRY

The platform shall maintain machine-readable event schemas.

Initial location:

```
/docs/events/
```

Example:

```
model_version_created.v1.json
element_modified.v1.json
geometry_generated.v1.json
validation_completed.v1.json
drawing_generated.v1.json
document_package_generated.v1.json
```

---

41. EVENT CONTRACT VALIDATION

Published events must validate against their registered schema.

Invalid events must not be published.

A malformed event represents an integration failure and must enter the appropriate failure path.

---

42. EVENT COMPATIBILITY

Event schema changes shall follow compatibility rules.

Compatible changes may include:

```
new optional fields
additional metadata
non-breaking enumeration additions where consumers tolerate them
```

Breaking changes require:

```
new event version
```

---

43. INTEGRATION ADAPTERS

External systems must connect through adapters.

Examples:

```
IFC Adapter
PDF Adapter
BIM Adapter
Cloud Storage Adapter
Webhook Adapter
Email Adapter
Professional Application Adapter
Municipal Integration Adapter
```

External integrations must not directly modify OneDraft database tables.

---

44. WEBHOOK SYSTEM

OneDraft may expose outbound webhooks for selected events.

Example:

```
POST /api/v1/integrations/webhooks
```

Webhook configuration contains:

```
id
organization_id
endpoint
event_types
status
secret_reference
```

```
created_at
updated_at
```

---

45. WEBHOOK DELIVERY

Webhook delivery follows:

```
EVENT
  ↓
WEBHOOK QUEUE
  ↓
HTTP DELIVERY
  ↓
ACKNOWLEDGEMENT
```

Webhook delivery is asynchronous.

A failed webhook must not cause the originating project transaction to fail.

---

46. WEBHOOK SIGNING

Outbound webhook requests shall be cryptographically signed.

Conceptually:

```
X-OneDraft-Event-ID
X-OneDraft-Event-Type
X-OneDraft-Signature
```

The receiving integration verifies the signature before accepting the event.

---

47. WEBHOOK RETRY

Initial retry strategy:

```
attempt 1
attempt 2
attempt 3
attempt 4
attempt 5
```

Retries use increasing delay.

After the configured retry limit, the delivery enters:

```
DEAD_LETTERED
```

The project state remains unaffected.

---

48. INBOUND INTEGRATIONS

Inbound external events must pass through an integration boundary.

Conceptually:

```
EXTERNAL SYSTEM
 ↓
INTEGRATION ADAPTER
 ↓
AUTHENTICATION
 ↓
SCHEMA VALIDATION
 ↓
NORMALIZATION
 ↓
DOMAIN COMMAND
 ↓
PROJECT MODEL
```

External events must not bypass domain validation.

---

49. INTEGRATION IDENTIFIERS

External references should be stored separately from internal UUIDs.

Conceptually:

```
system.external_references
```

Fields:

```
id UUID PRIMARY KEY
organization_id UUID NOT NULL
provider VARCHAR(128) NOT NULL
external_type VARCHAR(128) NOT NULL
external_id VARCHAR(512) NOT NULL
internal_type VARCHAR(128) NOT NULL
internal_id UUID NOT NULL
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique provider references prevent duplicate mappings.

---

50. INTEGRATION JOBS

Long-running integrations use the existing Job & Worker Execution System.

Example:

```
IFC IMPORT
 ↓
JOB
 ↓
ARTIFACT INGESTION
 ↓
MODEL TRANSLATION
 ↓
VALIDATION
 ↓
PROJECT MODEL UPDATE
 ↓
EVENTS
```

The event bus coordinates the lifecycle.

---

51. IMPORT PIPELINE

The initial import architecture supports:

```
PROJECT IMPORT
IFC IMPORT
CAD IMPORT
SURVEY IMPORT
DOCUMENT IMPORT
SCHEDULE IMPORT
```

Imported information must be normalized into the OneDraft domain model.

---

52. EXPORT PIPELINE

The initial export architecture supports:

```
PDF
IFC
CAD
CSV
JSON
PROJECT ARCHIVE
```

Exports are generated from a specified:

```
revision
model version
```

rather than the uncontrolled current state.

---

53. EXPORT EVENTS

Example:

```
EXPORT_REQUESTED
EXPORT_STARTED
EXPORT_GENERATED
EXPORT_VALIDATED
EXPORT_FAILED
```

Each export identifies its source model version.

---

54. ARTIFACT EVENT CHAIN

A generated document follows:

```
DOCUMENT_GENERATION_REQUESTED
        ↓
JOB_CREATED
        ↓
JOB_STARTED
        ↓
DRAWINGS_GENERATED
        ↓
DOCUMENT_PACKAGE_GENERATED
        ↓
ARTIFACT_CREATED
        ↓
ARTIFACT_HASHED
        ↓
ARTIFACT_VALIDATED
```

The resulting artifact becomes traceable to the originating project state.

---

55. MODEL CHANGE EVENT CHAIN

A model modification follows:

```
ARIA_COMMAND_PROPOSED
        ↓
ARIA_COMMAND_VALIDATED
        ↓
ARIA_COMMAND_EXECUTED
        ↓
MODEL_VERSION_CREATED
```

```
        ↓
REVISION_CREATED
        ↓
GEOMETRY_JOB_QUEUED
        ↓
GEOMETRY_GENERATED
        ↓
VALIDATION_REQUESTED
```

The downstream systems react independently.

---

56. REDLINE EVENT CHAIN

A redline lifecycle follows:

```
REDLINE_CREATED
        ↓
ARIA_COMMAND_PROPOSED
        ↓
ARIA_COMMAND_VALIDATED
        ↓
ARIA_COMMAND_EXECUTED
        ↓
MODEL_VERSION_CREATED
        ↓
DRAWING_INVALIDATED
        ↓
DRAWING_GENERATION_STARTED
        ↓
DRAWING_GENERATED
        ↓
REDLINE_RESOLVED
```

This creates a complete trace from human instruction to resulting documentation.

---

57. COMPLIANCE EVENT CHAIN

Compliance processing follows:

```
CODE_SOURCE_REGISTERED
        ↓
CODE_RULES_AVAILABLE
        ↓
CODE_MANIFEST_CREATED
        ↓
VALIDATION_REQUESTED
        ↓
VALIDATION_STARTED
        ↓
VALIDATION_COMPLETED
```

The manifest becomes part of the provenance chain for finalized documentation.

---

58. EVENT-DRIVEN INVALIDATION

Derived artifacts must become invalid when their source model state changes.

For example:

```
MODEL_VERSION_CHANGED
        ↓
AFFECTED GEOMETRY INVALIDATED
        ↓
AFFECTED DRAWINGS INVALIDATED
        ↓
AFFECTED VALIDATION INVALIDATED
        ↓
DOCUMENT PACKAGE INVALIDATED
```

The invalidation system must identify affected artifacts rather than blindly regenerating the entire project.

---

59. DEPENDENCY GRAPH

The event system works with the existing dependency graph:

```
PROJECT
 ↓
MODEL
 ↓
ELEMENT
 ↓
GEOMETRY
 ↓
VIEW
 ↓
DRAWING
 ↓
SHEET
 ↓
DOCUMENT
```

A change propagates through known dependencies.

---

60. SELECTIVE RECOMPUTATION

OneDraft shall avoid unnecessary recomputation.

Example:

```
Door parameter changed
```

should not automatically require regeneration of every unrelated project artifact.

The dependency system determines:

```
affected geometry
affected views
affected drawings
affected schedules
affected validation
affected documents
```

Only affected downstream artifacts are scheduled.

---

61. EVENT PRIORITY

Events may receive priority:

```
CRITICAL
HIGH
NORMAL
LOW
BACKGROUND
```

Critical project-state events receive processing priority over nonessential analytics or notification events.

---

62. EVENT BACKPRESSURE

Consumers may become slower than producers.

The event system must therefore support backpressure.

Conceptually:

```
EVENT PRODUCER
      ↓
QUEUE
      ↓
CONSUMER CAPACITY
```

The producer must not assume that every consumer can process events synchronously.

---

63. CONSUMER SCALING

Consumers may scale horizontally.

Example:

```
validation-worker-1
validation-worker-2
validation-worker-3
```

All workers consume from the same logical consumer group where appropriate.

---

64. DEAD-LETTER QUEUE

Events that repeatedly fail processing enter:

```
Dead Letter Queue
```

A dead-letter record includes:

```
event_id
event_type
consumer
attempt_count
last_error
payload_reference
first_failed_at
last_failed_at
```

Dead-letter events must be observable and recoverable.

---

65. RETRY POLICY

Transient errors should be retried.

Examples:

```
temporary network failure
temporary service unavailable
database connection interruption
external API timeout
worker capacity exhaustion
```

Permanent semantic errors should not be endlessly retried.

---

66. FAILURE CLASSIFICATION

Failures are classified as:

```
TRANSIENT
PERMANENT
CONSTRAINT
AUTHORIZATION
VALIDATION
SCHEMA
INTEGRATION
SYSTEM
```

The retry policy depends upon the classification.

---

67. EVENT OBSERVABILITY

Every event should be traceable through:

```
event_id
correlation_id
causation_id
project_id
source
event_type
timestamp
consumer
processing_status
```

This allows an administrator or engineer to reconstruct execution flow.

---

68. DISTRIBUTED TRACE

The runtime layer should propagate:

```
request_id
trace_id
span_id
correlation_id
```

across:

```
API
services
workers
event bus
external integrations
```

This makes a distributed OneDraft operation observable as one execution trace.

---

69. AUDIT INTEGRATION

Important domain events shall also be represented in the audit system.

The event bus must not replace the authoritative audit record.

Instead:

```
DOMAIN EVENT
 ↓
EVENT BUS
 ↓
AUDIT CONSUMER
 ↓
IMMUTABLE AUDIT RECORD
```

The audit subsystem remains responsible for durable audit semantics.

---

70. SECURITY BOUNDARY

Events are trusted only within authenticated OneDraft infrastructure.

Every publisher and consumer must authenticate to the messaging infrastructure.

Services receive only the permissions required for their event topics.

---

71. EVENT AUTHORIZATION

Publishing permissions are separated by subsystem.

Example:

```
geometry service
    → geometry events

validation service
    → validation events

documentation service
    → documentation events
```

A geometry service must not arbitrarily publish acceptance events.

---

72. TENANCY ISOLATION

Every project event must carry sufficient tenancy context to prevent cross-organization data leakage.

At minimum where applicable:

```
organization_id
project_id
```

Consumers must enforce tenancy boundaries.

---

73. SENSITIVE EVENT PAYLOADS

Events should avoid unnecessary sensitive information.

Where possible, events contain:

```
identifiers
references
metadata
state transitions
```

rather than duplicating private source documents or large artifacts.

---

74. LARGE PAYLOAD STRATEGY

Large files and computational payloads are not transported directly through the event bus.

Instead:

```
ARTIFACT STORAGE
        ↓
ARTIFACT REFERENCE
        ↓
EVENT
```

Example:

```
{
  "artifact_id": "uuid",
  "storage_reference": "internal-reference",
  "content_hash": "sha256..."
}
```

---

75. EVENT TRANSACTIONALITY

The event bus does not become the source of truth for transactional model state.

The database and domain services remain authoritative.

Events communicate state transitions after those transitions are durably recorded.

---

76. EVENTUAL CONSISTENCY

Derived systems operate under controlled eventual consistency.

For example:

```
MODEL UPDATED
 ↓
EVENT PUBLISHED
 ↓
DRAWING SERVICE RECEIVES EVENT
 ↓
DRAWING REGENERATED
```

There may be a short interval in which the model is current while its derived drawing remains stale.

The system must explicitly represent this condition rather than silently presenting stale output as current.

---

77. CONSISTENCY STATUS

Derived objects may use states such as:

```
CURRENT
STALE
INVALID
GENERATING
VALIDATING
FAILED
```

This status allows clients to understand whether an artifact corresponds to the current project model.

---

78. EVENT PROCESSING GUARANTEE

The initial system shall target:

```
at-least-once delivery
```

with:

```
idempotent consumers
```

Exactly-once distributed processing is not required for the MVP.

---

79. EVENT DELIVERY GUARANTEE

A successful consumer acknowledgement means:

```
event processing completed successfully
```

A failed acknowledgement results in retry according to the configured retry policy.

---

80. EVENT BUS TECHNOLOGY ABSTRACTION

The domain layer must not depend directly upon a specific messaging provider.

The architecture shall define an internal interface such as:

```
EventPublisher
EventConsumer
EventEnvelope
EventRouter
EventSerializer
```

The underlying implementation may use a message broker appropriate to the deployment environment.

---

81. EVENT PUBLISHER INTERFACE

Conceptually:

```
class EventPublisher:

    async def publish(
        self,
        event: EventEnvelope
    ) -> None:
        ...
```

The application layer depends on this interface rather than a concrete broker implementation.

---

82. EVENT CONSUMER INTERFACE

Conceptually:

```
class EventConsumer:

    async def handle(
        self,
        event: EventEnvelope
    ) -> None:
        ...
```

Consumers remain independently deployable.

---

83. EVENT ROUTER

The router determines which consumers receive which events.

Conceptually:

```
event_type
      ↓
routing rules
      ↓
consumer groups
```

Routing configuration must remain separate from domain business logic.

---

84. EVENT SERIALIZATION

Events are serialized using a stable machine-readable representation.

The initial platform shall use:

```
JSON
```

The architecture must permit a future binary event format if performance requirements justify it.

---

85. API / EVENT RELATIONSHIP

REST remains the primary external synchronous API.

Events provide asynchronous internal coordination.

Therefore:

```
REST
 ↓
request / response
```

while:

```
EVENT BUS
 ↓
state propagation
```

These are complementary interfaces rather than competing architectures.

---

86. API COMMAND TO EVENT

Example:

```
POST /projects/{id}/aria/requests
```

may result in:

```
ARIA_REQUEST_RECEIVED
```

followed by:

```
ARIA_COMMAND_PROPOSED
```

The API response does not need to wait for all downstream work to finish.

---

87. JOB TO EVENT

A job may produce:

```
JOB_COMPLETED
```

which triggers:

```
GEOMETRY_GENERATED
```

or:

```
DRAWING_GENERATED
```

or:

```
DOCUMENT_PACKAGE_GENERATED
```

depending upon the job type.

---

88. EVENT-DRIVEN ORCHESTRATION

The Runtime Orchestrator may use events to coordinate multi-stage workflows.

Example:

```
COMMAND_EXECUTED
        ↓
GEOMETRY_REQUIRED
        ↓
GEOMETRY_COMPLETE
        ↓
VALIDATION_REQUIRED
        ↓
VALIDATION_COMPLETE
        ↓
DOCUMENTATION_REQUIRED
        ↓
DOCUMENTATION_COMPLETE
```

The orchestrator tracks workflow state while the event bus transports state transitions.

---

89. WORKFLOW CORRELATION

Every orchestration workflow receives:

```
workflow_id
correlation_id
```

The workflow state identifies:

```
current_stage
completed_stages
failed_stages
pending_stages
required_artifacts
```

---

90. EVENT-DRIVEN PROJECT LIFECYCLE

The complete OneDraft lifecycle becomes:

```
PROJECT CREATED
        ↓
REQUIREMENTS RECORDED
        ↓
MODEL CREATED
        ↓
ARIA / USER MODIFICATION
        ↓
MODEL VERSION CREATED
```

```
        ↓
GEOMETRY GENERATED
        ↓
COMPLIANCE APPLIED
        ↓
VALIDATION COMPLETED
        ↓
DOCUMENTATION GENERATED
        ↓
ARTIFACT PACKAGE CREATED
        ↓
CUSTOMER REVIEW
        ↓
REDLINE
        ↓
MODEL REVISION
        ↓
REVALIDATION
        ↓
FINAL DOCUMENT PACKAGE
        ↓
CUSTOMER ACCEPTANCE
```

Every major transition is represented in the event architecture.

---

91. INTEGRATION WITH THE PROJECT MODEL

The Project Model remains authoritative.

The event bus never directly mutates the Project Model.

Instead:

```
EVENT
 ↓
CONSUMER
 ↓
DOMAIN COMMAND
 ↓
PROJECT MODEL SERVICE
 ↓
TRANSACTION
```

This preserves the command boundary established by the previous architecture.

---

92. INTEGRATION WITH GEOMETRY

Geometry is derived from the versioned model.

Events identify:

```
project_id
model_version_id
element_id
```

The geometry engine verifies that the requested computation corresponds to the expected model version.

---

93. INTEGRATION WITH COMPLIANCE

Compliance processing identifies:

```
project_id
revision_id
model_version_id
code_manifest_id
```

A validation result generated from one code manifest must never silently become associated with another manifest.

---

94. INTEGRATION WITH DOCUMENTATION

Documentation consumers identify the exact:

```
model_version_id
revision_id
view_id
drawing_id
```

before generating derived artifacts.

This preserves the established documentation provenance model.

---

95. INTEGRATION WITH ARTIFACT STORAGE

Generated files are stored as artifacts.

Events contain artifact references and hashes rather than embedding entire files.

Example:

```
DOCUMENT_PACKAGE_GENERATED
        ↓
artifact_id
        ↓
content_hash
        ↓
storage_reference
```

---

96. INTEGRATION WITH ACCEPTANCE

Acceptance can only reference a document package whose provenance is complete.

The acceptance workflow verifies:

```
project
revision
model version
code manifest
validation
document package
artifact hash
```

before recording final acceptance.

---

97. EVENT BUS DATABASE MIGRATIONS

The next persistence migrations include:

```
026_event_outbox
027_event_consumers
028_external_references
029_webhook_integrations
030_webhook_deliveries
031_event_indexes
032_event_seed_data
```

The exact numbering remains subject to the repository's migration state.

---

98. EVENT INDEX STRATEGY

High-frequency event queries receive indexes on:

```
event_id
event_type
project_id
aggregate_id
correlation_id
```

```
causation_id
status
created_at
published_at
```

Consumer tables receive indexes on:

```
consumer_name
event_id
status
processed_at
```

---

99. EVENT ARCHIVE

Long-term event retention may eventually move historical events to an archival store.

The live event infrastructure should not become indefinitely burdened by historical operational volume.

Archived domain events must remain retrievable by:

```
project_id
event_id
correlation_id
date range
```

---

100. EVENT RECONSTRUCTION TEST

OneDraft must eventually pass the following test:

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Correlation_ID
```

the system must be able to identify the event chain responsible for producing the associated derived artifacts.

The resulting chain should identify:

```
originating command
model transition
geometry computation
validation
drawing generation
document generation
acceptance
```

where applicable.

101. INTEGRATION TEST

The first event-bus integration test shall execute:

1. Create organization
2. Create user
3. Create project
4. Publish PROJECT_CREATED
5. Create model
6. Publish MODEL_CREATED
7. Create building
8. Publish BUILDING_CREATED
9. Create level
10. Publish LEVEL_CREATED
11. Create architectural elements
12. Publish ELEMENT_CREATED
13. Submit Aria command
14. Publish ARIA_COMMAND_PROPOSED
15. Execute command
16. Publish ARIA_COMMAND_EXECUTED
17. Create model revision
18. Publish MODEL_VERSION_CREATED
19. Queue geometry job
20. Publish GEOMETRY_JOB_QUEUED
21. Generate geometry
22. Publish GEOMETRY_GENERATED
23. Run validation
24. Publish VALIDATION_COMPLETED
25. Generate drawings
26. Publish DRAWING_GENERATED

```
27. Generate document package

28. Publish DOCUMENT_PACKAGE_GENERATED

29. Store artifact

30. Publish ARTIFACT_CREATED

31. Record acceptance

32. Publish ACCEPTANCE_RECORDED

33. Retrieve complete correlated event chain
```

This becomes the primary integration regression test.

---

102. FAILURE INTEGRATION TEST

The system must also test:

```
event publication failure
consumer failure
duplicate delivery
out-of-order delivery
worker failure
network failure
schema mismatch
dead-letter processing
consumer recovery
event replay
```

The project model must remain transactionally correct through each scenario.

---

103. DUPLICATE EVENT TEST

A consumer shall receive the same event twice.

Expected result:

```
first delivery → processed
second delivery → acknowledged as duplicate
```

No duplicate domain mutation may occur.

---

104. STALE EVENT TEST

An event referencing an obsolete model version shall be received.

The consumer must identify:

```
expected model version
actual current model version
```

and determine whether the operation is:

```
valid
stale
invalid
retriable
```

No stale derived artifact may silently become current.

---

105. OUT-OF-ORDER EVENT TEST

Where ordering is required, the consumer shall reject or defer an event whose prerequisite event has not yet been processed.

Example:

```
MODEL_VERSION_ACTIVATED
```

cannot become effective before:

```
MODEL_VERSION_CREATED
```

has been recorded.

---

106. DEAD-LETTER RECOVERY TEST

A failed event shall enter the dead-letter queue after the configured retry limit.

An operator or recovery service may then:

```
inspect
correct
retry
replay
resolve
```

the event.

Recovery must preserve the original event identifier.

---

107. WEBHOOK INTEGRATION TEST

The integration test shall verify:

```
domain event
 ↓
webhook event
 ↓
signed HTTP request
 ↓
receiver
 ↓
acknowledgement
```

and verify retry behavior when the receiver is unavailable.

---

108. SECURITY TEST

The integration layer shall verify:

```
unauthorized publisher rejected
unauthorized consumer rejected
invalid webhook signature rejected
cross-organization event rejected
invalid event schema rejected
expired credential rejected
```

---

109. OBSERVABILITY TEST

A single user operation shall be traceable through:

```
request_id
trace_id
correlation_id
event_id
job_id
revision_id
model_version_id
artifact_id
```

The engineering team must be able to reconstruct the operation without manually correlating unrelated logs.

---

110. EVENT BUS SUCCESS CRITERIA

The Integration & Event Bus implementation is successful when it can:

Publish domain events

Persist events through the outbox pattern

Deliver events asynchronously

Guarantee at-least-once delivery

Support idempotent consumers

Route events to appropriate services

Maintain correlation identifiers

Maintain causation identifiers

Support retries

Support dead-letter handling

Support event schema validation

Support webhook delivery

Support external references

Support artifact references

Support event replay

Support event-driven invalidation

Support distributed tracing

Preserve project provenance

---

111. SYSTEM-LEVEL SUCCESS CRITERIA

The entire OneDraft architecture must now support:

```
CLIENT
 ↓
API
 ↓
COMMAND
 ↓
DOMAIN SERVICE
 ↓
TRANSACTION
 ↓
```

```
OUTBOX
 ↓
EVENT BUS
 ↓
WORKERS / SERVICES
 ↓
DERIVED STATE
 ↓
ARTIFACTS
```

The event bus must never replace the domain model.

It connects the domain model to the computational systems that act upon it.

---

112. ARCHITECTURAL INVARIANT

The following invariant is established:

```
DATABASE
    =
AUTHORITATIVE TRANSACTIONAL STATE

EVENT BUS
    =
ASYNCHRONOUS STATE PROPAGATION

WORKERS
    =
COMPUTATIONAL EXECUTION

ARTIFACT STORAGE
    =
GENERATED OBJECT STORAGE

AUDIT
    =
IMMUTABLE ACTIVITY RECORD

API
    =
EXTERNAL MACHINE INTERFACE

ARIA
    =
INTELLIGENCE / COMMAND PROPOSAL LAYER
```

No subsystem is permitted to silently assume another subsystem's responsibility.

---

113. ONE-DIRECTIONAL AUTHORITY

The architecture therefore follows:

```
USER / CLIENT
      ↓
API
      ↓
COMMAND
      ↓
DOMAIN
      ↓
TRANSACTION
      ↓
EVENT
      ↓
DERIVED SYSTEM
```

Derived systems may request new commands, but they cannot redefine authoritative project state independently.

---

114. EVENT BUS AS SYSTEM NERVOUS SYSTEM

The Integration & Event Bus becomes the asynchronous coordination layer of OneDraft.

It allows the platform to operate as:

```
ONE SYSTEM
```

while remaining composed of:

```
MULTIPLE SERVICES
MULTIPLE WORKERS
MULTIPLE ENGINES
MULTIPLE STORAGE SYSTEMS
MULTIPLE INTEGRATIONS
```

without collapsing those components into a single monolithic implementation.

---

115. IMPLEMENTATION ARTIFACTS

The next engineering package shall consist of:

Artifact A

`026_event_outbox.sql`

Transactional event outbox and event persistence structures.

Artifact B

`event_contracts.py`

Python event envelopes, event types, schemas and serialization models.

Artifact C

`event_bus.py`

Internal event publisher, consumer and routing interfaces.

Artifact D

`outbox_publisher.py`

Transactional outbox polling and publication service.

Artifact E

`event_consumers.py`

Consumer registration and idempotent processing framework.

Artifact F

`webhook_service.py`

Outbound signed webhook delivery system.

Artifact G

`integration_service.py`

External-system integration boundary.

Artifact H

`event_replay_service.py`

Controlled historical event replay and reconstruction service.

---

116. REPOSITORY STRUCTURE

The implementation shall extend the existing OneDraft repository with:

```
apps/
    api/
    worker/
    orchestrator/
```

```
packages/
    domain/
    geometry/
    compliance/
    validation/
    documentation/
    runtime/
    jobs/
    events/
    integrations/
    artifacts/

docs/
    openapi/
    events/
    architecture/

migrations/
    026_event_outbox.sql
    027_event_consumers.sql
    028_external_references.sql
    029_webhook_integrations.sql
    030_webhook_deliveries.sql
    031_event_indexes.sql
```

The exact repository location may be adapted to the established implementation tree without changing the architecture.

---

117. INITIAL EVENT CONTRACT

The first machine-readable event contract shall include:

```
PROJECT_CREATED
MODEL_VERSION_CREATED
REVISION_CREATED
ARIA_COMMAND_PROPOSED
ARIA_COMMAND_EXECUTED
GEOMETRY_GENERATED
VALIDATION_COMPLETED
DRAWING_GENERATED
DOCUMENT_PACKAGE_GENERATED
ARTIFACT_CREATED
REDLINE_CREATED
REDLINE_RESOLVED
ACCEPTANCE_RECORDED
```

These events establish the minimum viable project lifecycle.

---

118. INITIAL EVENT BUS FLOW

The first operational flow becomes:

```
CREATE PROJECT
      ↓
PROJECT_CREATED
      ↓
CREATE MODEL
      ↓
MODEL_VERSION_CREATED
      ↓
ARIA COMMAND
      ↓
ARIA_COMMAND_EXECUTED
      ↓
REVISION_CREATED
      ↓
GEOMETRY_GENERATED
      ↓
VALIDATION_COMPLETED
      ↓
DRAWING_GENERATED
      ↓
DOCUMENT_PACKAGE_GENERATED
      ↓
ARTIFACT_CREATED
      ↓
ACCEPTANCE_RECORDED
```

This is the first complete event-driven OneDraft execution chain.

---

119. CURRENT ARCHITECTURAL STATE

OneDraft now contains:

Product Definition

System Architecture

Technology Architecture

Domain Model

Persistence Layer

API Contract

Project Model

Parametric Geometry Engine

Compliance & Regulatory Engine

Validation Engine

Revision System

Documentation & Drawing Generation Engine

Runtime Orchestrator

Job & Worker Execution System

Artifact Storage

Integration & Event Bus

AI Command Boundary

Document Provenance

Customer Acceptance Workflow

Audit Architecture

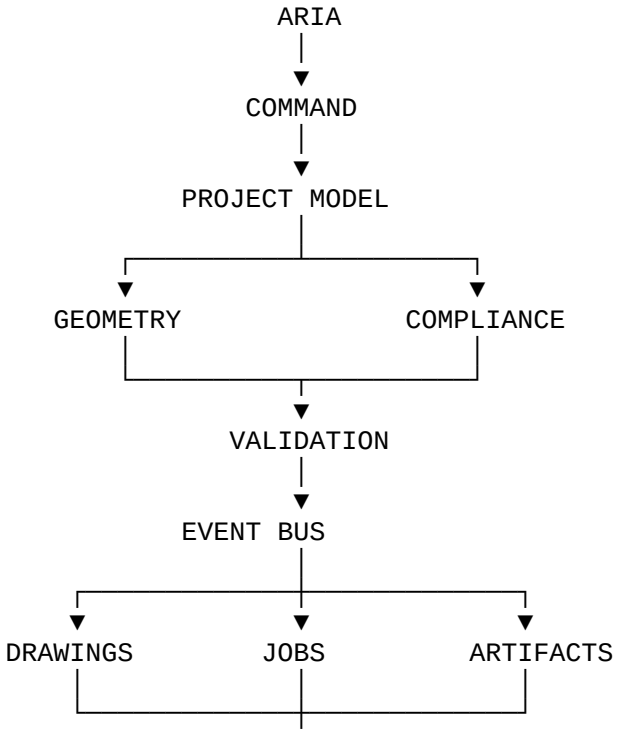
The platform has therefore progressed from a collection of subsystem specifications into an interconnected computational architecture.

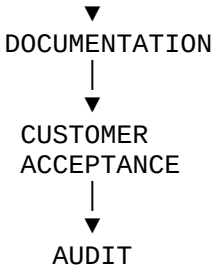
---

120. FINAL ARCHITECTURAL DECISION

OneDraft shall not operate as a collection of independently executing tools.

It shall operate as a coordinated computational system:





The Event Bus provides the coordination mechanism while the Project Model remains the authoritative computational state.

---

121. SPECIFICATION STATUS

ONEDRAFT INTEGRATION & EVENT BUS v0.1

STATUS: ESTABLISHED

The Integration & Event Bus architecture is now defined as the asynchronous communication and coordination layer between the established OneDraft subsystems.

The next implementation stage is the creation of the executable event infrastructure:

`026_event_outbox.sql`

`event_contracts.py`

`event_bus.py`

`outbox_publisher.py`

`event_consumers.py`

`webhook_service.py`

`integration_service.py`

`event_replay_service.py`

These artifacts will convert the Integration & Event Bus specification into executable infrastructure.